

CƠ SỞ LẬP TRÌNH (ITS318) — CHƯƠNG 7

Kiểu Chuỗi Ký Tự

Khi Máy Tính Bắt Đầu "Đọc Chữ"

Giảng viên: Phó Hải Đăng

Hành Trình Buổi Học Hôm Nay

01

SCQA Hook

Từ bài toán mật khẩu Wi-Fi đến xử lý chuỗi

02

Khai Báo & Bản Chất

Chuỗi = Mảng ký tự + ký tự kết thúc '\0'

03

Nhập / Xuất Chuỗi

scanf vs fgets — chọn vũ khí phù hợp

04

Thư Viện string.h

Bộ công cụ vàng: Đo - Sánh - Nối - Chép - Tìm

05

Thực Hành & Cầu Nối

Lab 3 mức + tổng kết + bridge sang Chương 8

01

SCQA HOOK

Khởi động từ bài toán thực tế

Thế Giới Của "Chữ" — Từ Mật Khẩu Đến Chatbot

01 Ngân Hàng

Xác thực tên chủ tài khoản, CMND/CCCD, mã OTP — tất cả đều là chuỗi ký tự

02 Tìm Kiếm

Google xử lý 8.5 tỷ truy vấn/ngày — mỗi truy vấn là một chuỗi cần phân tích

03 AI & Chat

Mọi prompt gõ cho ChatGPT, mọi tin nhắn Zalo — đều là chuỗi ký tự cần xử lý

Chuỗi Ký Tự Trong C — Không Có "Miễn Phí"

NGHỊCH LÝ: C KHÔNG CÓ KIỂU STRING!

- Python, Java có kiểu String sẵn, rất dễ dùng
- C thì không! Chuỗi trong C chỉ là mảng char kết thúc bằng '\0'
- Không thể gán `str1 = str2`, không thể so sánh bằng `==`
- Quên '\0' dẫn đến buffer overflow — lỗ hổng bảo mật nguy hiểm

TẠI SAO C LÀM VẬY?

C là ngôn ngữ "bậc thấp", gần phần cứng. Nó cho lập trình viên toàn quyền kiểm soát bộ nhớ. Hiểu chuỗi trong C = hiểu sâu bản chất máy tính hoạt động.

Vậy Làm Sao Để "Thuần Hóa" Chuỗi Trong C?

1

Khai báo và khởi tạo chuỗi đúng cách như thế nào?

2

Nhập chuỗi từ bàn phím — scanf hay fgets, cái nào an toàn hơn?

3

So sánh, nối, tìm kiếm, cắt chuỗi — dùng hàm gì?

Thư viện string.h chính là bộ công cụ vàng giúp các em làm chủ mọi thao tác với chuỗi!

02

KHAI BÁO & BẢN CHẤT CHUỖI

Chuỗi = Mảng Ký Tự + '\0'

Bản Chất Chuỗi: Mảng Ký Tự Có "Dấu Chấm Hết"

CÔNG THỨC CỐT LÕI: Chuỗi ký tự = Mảng 1 chiều kiểu char + ký tự kết thúc '\0' (NULL)

char Greeting[]

Index:

'H'	'E'	'L'	'L'	'O'	'\0'
0	1	2	3	4	5

char HoTen[30]; → Lưu tối đa 29 ký tự + 1 '\0'

Bảng mã ASCII: A=65, a=97, 0=48, '\0'=0

Ký tự '\0' chiếm 1 ô nhớ → luôn khai báo n+1

3 Cách Khai Báo Chuỗi — Chọn Cách Nào?

1

Cách 1: Tường Minh

```
char Greeting[] = {'H','E','L','L','O','\0'};
```

Ghi rõ từng ký tự + '\0' cuối cùng

2

Cách 2: Có Kích Thước

```
char Greeting[6] = "HELLO";
```

Chỉ rõ kích thước mảng (n+1)

3

Cách 3: Tự Động

```
char Greeting[] = "HELLO";
```

Compiler tự tính kích thước — gọn nhất

strlen() vs sizeof — Hai Thước Đo Khác Nhau

strlen(str)

→ Trả về độ dài chuỗi

(số ký tự TRƯỚC '\0')

→ Cần: #include <string.h>

VD: char name[30] = "Dang";

→ strlen(name) = 4

sizeof(str)

→ Trả về kích thước mảng

(số byte được cấp phát)

→ Không cần thư viện

VD: char name[30] = "Dang";

→ sizeof(name) = 30

Ẩn dụ: Căn phòng 30m² nhưng chỉ bày đồ 4m². strlen = diện tích đồ (4), sizeof = diện tích phòng (30)

Sai lầm phổ biến: Nhầm strlen và sizeof khi duyệt mảng hoặc cấp phát bộ nhớ!

Nền Tảng: Chuỗi Đứng Trên Vai "Mảng" Và "Con Trỏ"

1

Chương 5: Mảng

Mọi kỹ thuật duyệt mảng
đều áp dụng được: vòng
for, while, chỉ số [i]



2

Chương 6: Con Trỏ

Tên chuỗi = con trỏ tới
phần tử đầu tiên. str
tương đương &str[0]



3

Ứng dụng ngay

scanf("%s", str) không
cần dấu & — vì str ĐÃ LÀ
địa chỉ rồi!

03

NHẬP / XUẤT CHUỖI

scanf vs fgets — chọn vũ khí phù hợp

scanf("%s") — Nhập Chuỗi Nhưng Sợ Dấu Cách

ĐẶC ĐIỂM SCANF

Cú pháp: `scanf("%s", str);`

Lưu ý: KHÔNG cần dấu & (str đã là con trỏ)

Dừng đọc khi gặp khoảng trắng/tab/Enter

Chỉ đọc được MỘT từ duy nhất

```
#include <stdio.h>
int main() {
    char ten[20];
    printf("Nhap ten: ");
    scanf("%s", ten);
    printf("Ten: %s", ten);
    return 0;
}
```

Demo: Nhập "Nguyen Van A" → str chỉ chứa "Nguyen" (phần "Van A" bị bỏ lại trong bộ đệm!)

Nguy hiểm: Không kiểm soát được độ dài chuỗi nhập → có thể tràn bộ đệm (buffer overflow)

fgets() — Hàm "Kiên Nhẫn" Đọc Trọn Vẹn Chuỗi

ĐẶC ĐIỂM FGETS

`fgets(str, sizeof(str), stdin);`

- Đọc tối đa n-1 ký tự hoặc đến Enter
- An toàn: kiểm soát được độ dài tối đa
- **Lưu ý:** giữ lại ký tự '\n' cuối chuỗi

```
#include <stdio.h>
int main() {
    char name[30];
    printf("Nhap ten: ");
    fgets(name, sizeof(name), stdin);
    printf("Ten: %s", name);
    return 0;
}
```

Demo: Nhập "Nguyen Van A" → str chứa "Nguyen Van A\n" (đầy đủ, nhưng có '\n' cuối)

Hàm `gets()` đã bị loại bỏ khỏi C chuẩn vì không an toàn (không giới hạn độ dài) → Luôn dùng `fgets`!

scanf vs fgets — Chọn "Vũ Khí" Phù Hợp

Tiêu chí	scanf("%s")	fgets()
Đọc dấu cách	Không	Có
Giới hạn độ dài	Không tự động	Tham số n
An toàn bộ nhớ	Rủi ro tràn	An toàn
Giữ '\n' cuối	Không	Có (cần xử lý)
Dùng khi nào	Nhập 1 từ đơn giản	Nhập chuỗi có dấu cách

Kết luận: Trong bài tập và dự án, hãy ưu tiên fgets()!

Xuất Chuỗi Ra Màn Hình

```
printf("%s", str);
```

Xuất chuỗi, kết hợp format khác (%d, %f...)

```
fputs(str, stdout);
```

Xuất chuỗi đơn giản (không tự thêm '\n')

```
puts(str);
```

Xuất chuỗi + tự thêm '\n' cuối dòng

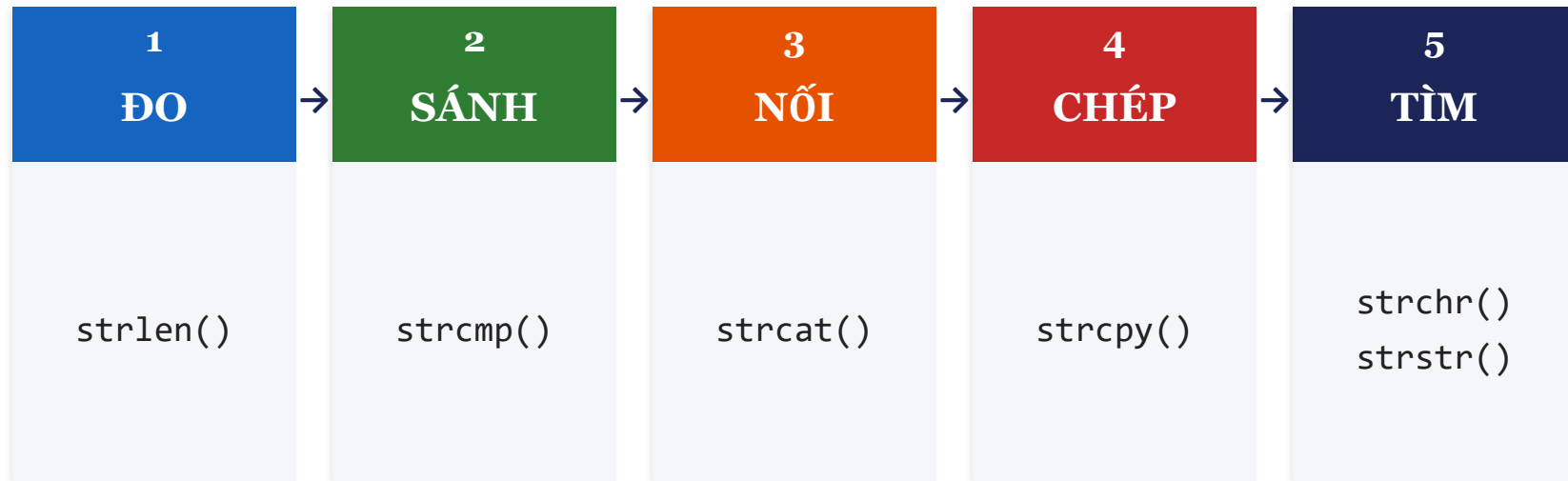
04

THƯ VIỆN STRING.H

Bộ công cụ vàng: Đo - Sánh - Nối - Chép - Tìm

string.h — Bộ Công Cụ Vàng Xử Lý Chuỗi

Bắt buộc: `#include <string.h>` ở đầu chương trình



Đo Và So — Hai Thao Tác Cơ Bản Nhất

strlen(str)

Trả về số ký tự (không tính '\0')

VD: strlen("Hello") = 5

Cần: #include <string.h>

strcmp(str1, str2)

Trả về 0 → giống nhau

Trả về < 0 → str1 "nhỏ hơn" str2

Trả về > 0 → str1 "lớn hơn" str2

stricmp(s1, s2)

So sánh KHÔNG phân biệt hoa/thường

strncmp(s1, s2, n)

Chỉ so sánh n ký tự đầu tiên

CẢNH BÁO: Không dùng == để so sánh chuỗi! (== so sánh địa chỉ, KHÔNG phải nội dung)

Nối & Chép — Biến Đổi Chuỗi

GHÉP NỐI CHUỖI

`strcat(str1, str2)`

Nối str2 vào cuối str1

str1 phải đủ chỗ chứa!

`strncat(str1, str2, n)`

Chỉ nối n ký tự đầu của str2

Quy tắc: đích trước, nguồn sau

SAO CHÉP CHUỖI

`strcpy(str1, str2)`

Copy toàn bộ str2 vào str1

str1 bị ghi đè hoàn toàn!

`strdup(str)`

Nhân bản chuỗi (tự cấp phát bộ nhớ)

Luôn đảm bảo str1 đủ kích thước

Tìm Kim Trong Đồng "Ký Tự"

```
strchr(str, ch)
```

Tìm ký tự ch xuất hiện ĐẦU TIÊN trong str

VD: `strchr(email, '@')` → kiểm tra email hợp lệ

```
strrchr(str, ch)
```

Tìm ký tự ch xuất hiện CUỐI CÙNG trong str

VD: `strrchr(path, '/')` → tách tên file từ đường dẫn

```
strstr(str1, str2)
```

Tìm chuỗi con str2 trong chuỗi str1

VD: `strstr(log, "error")` → tìm lỗi trong file log

Kết quả: Trả về con trỏ tới vị trí tìm thấy, hoặc NULL nếu không tìm thấy

Thêm Công Cụ: ctype.h Và stdlib.h

ctype.h — Xử lý ký tự đơn

toupper(ch) / tolower(ch)

Chuyển hoa/thường từng ký tự

isalpha(ch) → là chữ cái?

isdigit(ch) → là chữ số?

isspace(ch) → là khoảng trắng?

isupper(ch) / islower(ch)

stdlib.h — Chuỗi → Số

atoi(str) → int

atol(str) → long

atof(str) → double

VD: atoi("123") = 123

string.h bổ sung

strlwr(str) / strupr(str)

Chuyển chuỗi hoa/thường

strrev(str) → đảo ngược chuỗi

05

THỰC HÀNH & CẦU NỐI

Lab 3 mức + Tổng kết

Lab 1: Thực Hành Cơ Bản (Bắt buộc — 15 phút)

1

Tính strlen thủ công

Viết hàm tính độ dài chuỗi KHÔNG dùng strlen — duyệt bằng while đến '\0'

2

Tách ký tự

Nhập chuỗi, in ra từng ký tự trên mỗi dòng — duyệt mảng char

3

In ngược chuỗi

Nhập chuỗi, in ra theo chiều ngược lại — dùng strlen + vòng for đi ngược

Lab 2 (Nâng Cao) & Lab 3 (Thử Thách)

LAB 2 — NÂNG CAO (Khuyến khích — 20 phút)

Bài 4: Chuyển chuỗi hoa $\leftrightarrow$ thường (dùng toupper/tolower)

Bài 5: Đếm số từ trong chuỗi (đếm khoảng trắng + 1)

Bài 6: So sánh 2 chuỗi KHÔNG dùng strcmp (duyệt từng ký tự)

LAB 3 — THỬ THÁCH (Optional — Cộng điểm thưởng!)

Bài 7: Tìm ký tự xuất hiện nhiều nhất (mảng đếm tần suất ASCII)

Bài 8: Sắp xếp ký tự theo thứ tự ABC (Bubble Sort trên mảng char)

Bài 9: Tìm từ dài nhất và ngắn nhất trong chuỗi

Bài tập về nhà: Bài đề nghị 1-7 trong ebook (phần 7.7) — nộp trước buổi sau

Bản Đồ Chương 7 — Mọi Thứ Cần Nhớ

Chuỗi = Mảng char + '\0' | Nhập: fgets (an toàn) | Xử lý: Đo - Sánh - Nối - Chép - Tìm

5 LỖI KINH ĐIỂN CẦN TRÁNH

1 Quên '\0' khi khai báo thủ công → buffer overflow

2 Dùng == thay vì strcmp() để so sánh chuỗi

3 Dùng = thay vì strcpy() để gán chuỗi

4 Mảng đích không đủ chỗ khi gọi strcat/strcpy

5 Dùng gets() — đã bị loại bỏ, phải dùng fgets()

Từ Chuỗi Đơn Lẻ Đến "Bộ Dữ Liệu" Hoàn Chỉnh

HÔM NAY — Chương 7

Lưu trữ và xử lý MỘT thông tin dạng chữ:

```
char ten[30] = "Nguyen Van A";  
int tuoi = 20;  
float diem = 8.5;
```

→ Ba biến riêng lẻ, không liên kết



SẮP TỚI — Chương 8: struct

Gom tất cả vào MỘT biến duy nhất:

```
struct SinhVien {  
    char ten[30];  
    int tuoi;  
    float diem;  
};
```

→ Kiểu dữ liệu tùy chỉnh!

CHUỖI = MẢNG CHAR + '\0'

THƯ VIỆN STRING.H LÀ CHÌA KHÓA

Hẹn gặp lại lớp mình ở Chương 8

Kiểu Cấu Trúc (struct)!

Giảng viên: Phó Hải Đăng